GAME DESIGN DOCUMENT

Good Enough Repairs

Final GDD

Author	Nikola Bulog
Version	1.3
Date	October 23, 2026 (CEST)
Status	Final

About this document

This is the final GDD: it describes the smallest playable version of the game (the MVP), based on the game jam prototype, and what follows it until release. Bigger ideas are kept in a separate Expansion plan. All dates are in Central European Time.

Version	Date	Changes
1.0	Oct 16, 2026	First draft of the core concept
1.1	Oct 20, 2026	Added AI Architecture and Technical Strategy, updated mechanics
1.2	Oct 22, 2026	Changes from the first agent stress test and own review
1.3	Oct 23, 2026	Final GDD: changes from the second stress test and own review (see Appendix: Revision notes)

Executive Summary

Good Enough Repairs is a 2D repair-management game. Damaged ships come to your station, and you fix them mostly by swapping parts between ships instead of buying new ones. You win by surviving 7 game days and lose by going bankrupt.

Pitch: You are a space mechanic. Damaged fighters and freighters dock at your repair station, and you keep them flying - and keep yourself in business by cleverly shuffling parts between ships.

Working title	Good Enough Repairs (jam version: Space Repair)
Genre	Management / repair sim, light puzzle
Platform	PC (Unity)
Session length	A few minutes per game day (round); 7 game days per run, plus an optional tutorial day
Target player	Fans of short, clever management games (repair sims, shop sims)
Origin	Game jam prototype, theme "repair"
AI use	Development only: a team of 7 AI agents builds, balances and tests the game. No AI runs inside the game.
Team	One developer (the designer, up to 10 hours per week for decisions, reviews and playtests), working with 7 AI agents

Design principles - every new feature must support at least one of these:

1. **Resourcefulness beats spending.** The smart player wins by reusing parts, not by buying new ones.
2. **Good enough is good enough.** A ship's parts only need to reach the required level, not perfect.
3. **Three ships, one puzzle.** Three docked ships are an opportunity, not a burden: the player works with all their parts at once.

Setting & theme

Most space games are about the fighter or the freighter. This game is about the third, overlooked one: the mechanic who takes care of the other two when they get stranded.

- **Fighters** - combat ships. They arrive with battle damage.
- **Freighters (cargo ships)** - haulers. They arrive with wear and tear.
- **The mechanic (the player)** - the one who saves you when you're stranded. Underpaid, resourceful, proud of it, and a little cheeky.

Tagline: They think they are important and scary. Wait till the breakdown to see what is scary, and wait for the fix to see who is important.

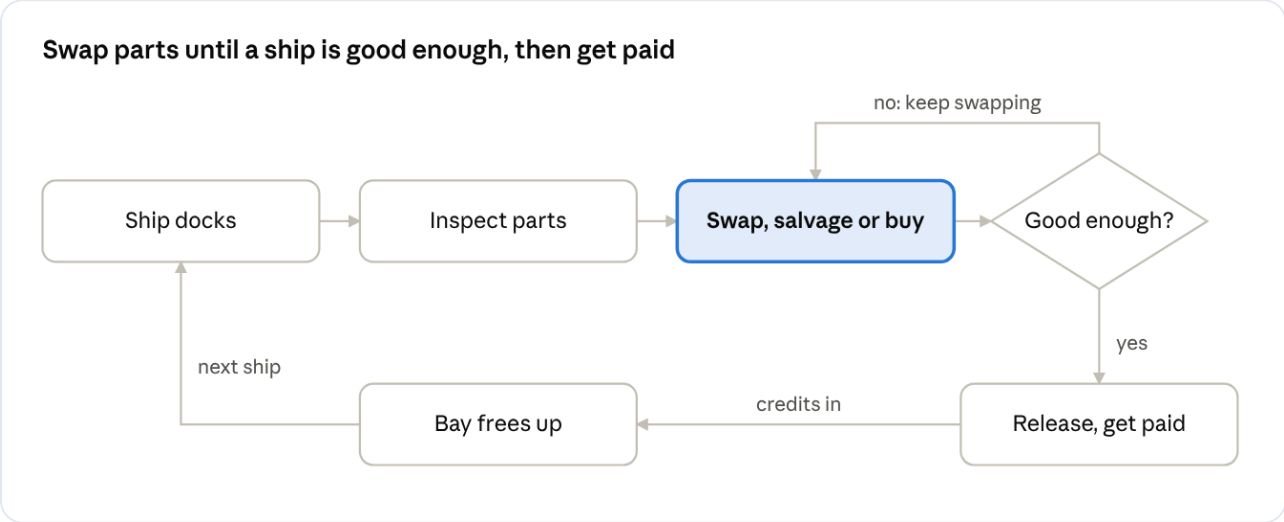
Tone: proud and cheeky. The mechanic enjoys squeezing the most out of every part, and the game's voice in hints and messages has the same grin. The mechanic speaks at the day start, in release warnings, on penalty fills, at the end of the day, on bankruptcy and at the end of the run.

Location (for v1): a static repair station. Whether the mechanic later travels on a mobile repair ship is left for the Expansion plan. The v1 core loop works the same either way, so it doesn't block the prototype.

Game Mechanics

Core gameplay loop

The player repeats one short loop for every ship, while juggling three bays at once.



core loop · one ship, repeated across 3 bays

The highlighted step is where the game happens: harvesting good parts from one ship to fix another is what keeps the player out of bankruptcy.

What the player sees

A day starts. A battered freighter slides in from the left and docks in the first bay, and a fighter follows into the second. Each bay shows the ship's payment and its weakest part. The player clicks the freighter: four parts pass the required level, but the engine is far below it at 35%. The fighter's engine is at 65%, not enough for a fighter but enough for a freighter. The player drags it across, drops the broken engine in the bin for a few credits, and now every part on the freighter passes. The freighter flies off to the right, coins land in the credit counter, and the next ship is already sliding in.

Docking bays

- The station has **3 docking bays**. Up to 3 ships are serviced at the same time, and each waits as long as needed.
- Ships enter the screen from one side and exit on the other. As soon as a released ship leaves the screen, the next ship enters.
- Each ship shows: type (fighter or freighter), payment offered, and required condition.

Ships and parts

- Each ship has a fixed set of **part slots** (e.g., engine, hull plate, shield generator, thruster, power core). Fighters and freighters use the same part types, so a part fits the matching slot on either ship.
- Each installed part has a **condition from 0-100%**, shown as a condition bar.
- There is no overall average: the weakest part decides. Ships arrive with 1-3 parts below the required level (the Economy Balancer sets how this rises with the difficulty ramp), which must be replaced with better ones *(the others can be swapped for worse parts, as long as they stay above the level)*.
- A ship is **repaired** when each of its parts is at or above the required level for that ship type: 60% on freighters, 70% on fighters. Parts never need to be at 100%.
- A ship can always be released. If a slot is empty or below the required level, a warning first shows the net result (payment minus penalty parts), and releasing fills that slot with a new part at the penalty price, 20% above the shop price, so buying the part yourself is always better.

Player actions

Action	What it does	Cost
Open ship view	Shows all part slots and their condition bars	Free
Move part	Drags a part in one move: from ship to ship, or to and from the inventory. Dropping it on an occupied slot swaps the two parts; dropping it on a full inventory doesn't work, and the part snaps back	Free
Salvage part	On the junkyard screen, one click takes a part into the inventory	Same price for every part
Bin part	Throws a part away for a small scrap payment	Pays a little
Buy part	A buy button next to each slot puts a brand-new 100% part straight into that slot, after a warning that shows the price. The button is only active when the slot is empty or its part is below the required level. The old part goes to the inventory; if the inventory is full, the warning says the old part will be binned	Expensive
Release ship	Sends the ship off and collects payment. Empty or too-weak slots are filled with new parts at the penalty price (20% above the shop price), after a warning	Payment, minus any filled parts

Inventory

- A small storage for removed parts, shared across all three bays *(10 slots)*.
- Parts stay in the inventory between ships and between days, so a part saved from an earlier repair can fix a later ship. Limited space forces decisions: keep a part for a future ship, or bin it. On a bad day, binning inventory parts for scrap can bring in the few credits that are needed.

Junkyard

- A screen opened from the day start screen before each game day, carried over from the jam version. It lists the 15 salvaged parts available that day.
- 7-10 of them are junk, unusable on any ship. The rest are good enough to use. As the mechanic's skill grows during the run, he finds more usable parts.
- The player salvages the parts they want with one click, and each goes into the inventory; if the inventory's 10 slots are full, the bin is on this screen too, so a part can be binned to make room.
- Every junkyard part costs the same, whatever its condition: always more than the bin pays (10), far less than a new part (150-300). So buying junk to bin it never makes money.

The secret strategy: part swapping

This is the heart of the game. Buying only new parts bankrupts you; the winning move is to **take good parts out of one ship and replace them with worse-but-acceptable parts, then use the good parts on another ship, and to dig the few good parts out of the junkyard.**

Worked example:

1. Freighters need every part at 60% or more, fighters at 70%. Freighter B arrives with a broken engine at 35%. Fighter A, docked next to it, has a 65% engine but a broken thruster.
2. Move A's 65% engine into B. Bin B's 35% engine for a little scrap money.
3. B now passes the required level, but its thruster is at 90%, far more than it needs. Swap it for a 62% thruster saved in the inventory from an earlier repair and keep the 90% one.
4. Release B and collect the payment. B's bay frees up for the next ship.
5. Put the 90% thruster into A. B was fine with a 62% thruster, but A is a fighter and needs 70%. A still needs an engine, so it waits: the junkyard or the next ship that docks in B's old bay may have one. Buying a new engine is the last resort.

Order matters: fix the easiest ship first. Releasing it frees a bay, and every new ship brings parts that might fix the harder ones.

Why it's fun: the player has to *see* the whole station as one puzzle, not three separate ships. The "aha" moment is realizing that a healthy part on one ship is money waiting to be used.

Design rules to protect part swapping:

- New parts must always be noticeably expensive compared to payment, so swapping is clearly better.
- Ships must regularly arrive with *some* parts above what they need, so there's always something to harvest.
- The game never explains the trick up front. In the tutorial, the mechanic drops a cheeky hint at a fixed step, while the fighter with the 65% engine is on screen: "That engine is too weak for a fighter, but a freighter would take it." Players who skipped the tutorial get a similar hint the first time their balance drops below the day's starting balance, pointing at a spare part that is actually on screen.

Economy & balancing

The economy is tuned so that **pure buying loses money and smart swapping makes money.** That gap is the whole game.

Item	Amount in credits	Why
Starting credits	500	Enough to survive a few mistakes (the Economy Balancer may lower it, even to 0)
New part price	150-300	Higher than most single-ship payments
Fighter payment	200-350	Pays more, but needs 70% parts
Freighter payment	150-250	Pays less
Station upkeep	120 per day	Constant pressure to keep earning
Scrap price (bin)	10 per part	Small reward for clearing the inventory, never worth more than a swap
Junkyard part price	Same for every part: above 10, far below 150	Cheap parts let salvaging and swapping beat buying, and binning them never pays
Penalty part price	New part price + 20%	Buying the part yourself is always cheaper than releasing an unfinished ship. Filling one weak part must cost more than the lowest ship payment

Balance check to run in playtests: a player who only buys new parts should go bankrupt by about day 3. A player who swaps well should end each day with slightly more money than they started, also on the hardest days.

Win/lose conditions & progression

- Tutorial: the first time the game starts, it asks "First time playing?" (tutorial) or "I know how to play". The tutorial is day 0: a guided quiet day with fixed ships and step-by-step instructions. It leads the player through swapping and can't be lost. There is no junkyard on day 0, since the player doesn't know yet what parts are for. After day 0 the credits and inventory are reset, and the real run starts at day 1. Players who skip it go straight to the main run.
- **Structure:** the game is played in game **days** (rounds). Each day is one of two kinds, and the start of the day tells the player which one and why. On a quiet day, a fixed number of ships (5-10) arrives, and the day ends when the last one flies away. On a busy day, more ships come than anyone could finish, so time is the only limit: ships keep arriving until a countdown (2-5 minutes) reaches 0, and the player tries to serve as many as possible. At 0 no new ships arrive, and the day ends when the last docked ship flies away, the same as on a quiet day. The two kinds are mixed, with more busy days later in the run. Upkeep is paid at the start of each day.
- **No timer per ship:** a docked ship waits as long as needed. The pressure comes from the day's limit and from money, not from individual ships leaving. Because a ship can always be released, every day can end.
- Money during the day: the credit counter changes live with every purchase, payment and scrap sale, and it may go below 0 and back up again. Only the balance at the end of the day counts: if it is below 0 then, the player is bankrupt. Holding the last ship back gains nothing, because the run only continues when the day ends.
- End of the run: after day 7, or after a bankruptcy, a final screen shows the run summary (credits earned, ships repaired, parts saved from the junkyard) and a closing line from the mechanic that depends on how the run went.
- **Lose:** credits are below 0 when the day ends → bankrupt, game over.
- **Win**: survive days 1-7. (Endless mode comes after the MVP.)
- **Score:** final credits minus starting credits.
- **Difficulty ramp:** as the mechanic's reputation grows, harder ships arrive, with more weak parts and lower conditions. Part prices stay fixed.

UI / screens

Screen	Shows	Main interactions
Station view (main)	3 docking bays, each ship's weakest part vs. the required level, payment, credits, time or ships left in the day	Click a ship to open it
Ship view	All part slots with condition bars. Opens over its own bay, and all three can be open at once	Drag parts in and out, or straight to another open ship; release ship
Inventory panel	Stored parts with condition	Drag to a ship slot, or to the bin
Shop	No separate screen: a buy button next to each slot shows the new part's price	Buy
End-of-day summary	Earned, spent, upkeep, balance	Continue / game over
Junkyard	Opened from the day start screen: the parts available that day, with condition, next to the inventory	Salvage any number of parts with one click each, bin inventory parts to make room if needed, then start the day
Day start	The day type (quiet or busy day) and the reason for it	Open the junkyard, start the day
Run end	Run summary: credits earned, ships repaired, parts saved from the junkyard; a closing line from the mechanic	Play again

Key UI requirement: the player must compare parts across ships quickly. All three ship views and the inventory can be open at the same time, so dragging a part from ship A to ship B is a single move.

Art & audio direction

Keep it simple for v1; readability matters more than detail. This section says what the player must understand at each moment; the UX Designer decides how to show it (animations, effects, tooltips, messages).

- **Visual style:** 2D. Condition bars are colored against the required level of the ship the part is in: red below it, green when it passes, and a third color (blue or purple) for a surplus part worth harvesting. The UX Designer makes a few versions of the colors and the bar's look for the designer to choose from. Parts in the inventory and junkyard show marks at 60% and 70%.
- **Mood:** the mechanic's station isn't shiny, but it is rugged and meticulously kept: worn metal, patched panels, every tool in its place.
- **Feedback:** effects match the quality of the move: a clean snap for an install, a satisfying sound and coins for a release that makes money, and a flat sound with no coins for a release that loses money. For example, a floating "+34" or "-12" rises and fades when a ship is released. The player should also understand that the credit counter can go below 0 during the day and only the end-of-day balance counts. No alarms: a countdown (time left or ships left) shows how much of the day remains, and a banner appears when it reaches 0 to signal that no new ships will come.
- **Music:** calm, looping background that speeds up near the end of a day.

AI Architecture

The game has no AI at runtime: every rule runs in normal game code. AI is used to **build** the game, with a team of seven development agents. Each agent owns one part of the game the player touches, so each one is described below by what the player experiences.

What each agent does

Agent	Role	Starts when	Effect on gameplay
Director	Turns the GDD into feature cards on a feature board, gives each agent its subtasks, rejects anything outside the scope, and takes open questions to the designer.	A work session starts, or QA reports on a feature	The player gets a small, finished game: every feature traces back to the core loop.
Programmer	Writes all the game logic: docking bays, parts, inventory, junkyard, the player actions, and the events other agents connect to.	A feature card has a coding subtask	Moving a part between ships is instant and reliable, and a ship can always be released.
Economy Balancer	Sets prices, payments, scrap value, upkeep and junkyard contents, and decides how damaged each arriving ship is.	A card needs balance numbers, or the simulation fails	A player who only buys new parts goes bankrupt by about day 3, and ships usually arrive with a healthy part worth harvesting.
UX Designer	Decides how the game responds to every player action, then places the art and sound in the scene and connects them to the Programmer's events. When a moment is confusing in screenshots or playtests, it adds or proposes effects, messages and warnings. It also writes the mechanic's lines.	A card needs a layout, or art and sound placed; or a feature is finished and needs a review	The player sees at a glance which parts are spare, moves one in a couple of clicks, and always knows how much of the day is left.
Artist	Makes the 2D art for the ships, parts, station, junkyard and icons, and imports it into Unity.	A card asks for a new image	Condition bars read at a glance, and the station looks rugged but well kept.
Sound Designer	Makes or picks the sound effects and music, and imports them into Unity.	A card asks for a new sound	Every install clunks, every payment rings, and the music tells the player the day is ending.
QA Playtester	Tests each finished feature in the build, and builds and runs the buyer/swapper simulation.	Every subtask of a feature card is done, or balance numbers change	No lost parts or stuck ships, and no exploit that makes the game trivially easy.

How each agent works

Agent	Reads	Writes (format)	Model or tool
Director	Whole GDD, feature board, QA reports	Feature board: cards with subtasks (Markdown)	Large model
Programmer	Game Mechanics and UI sections, its subtask, UX plan, game code	Logic scripts (C#) and logic objects in scenes and prefabs	Large model
Economy Balancer	Economy section, simulation results	One balance data file (JSON): prices, payments, damage, junkyard contents	Fast, cheaper model
UX Designer	Game Mechanics, UI and Art & audio sections, screenshots of the build	Screen layouts and feedback list (Markdown), visual parts of scenes and prefabs	Fast model; large model if it makes too many mistakes
Artist	Art direction, UX feedback list	Sprites and icons (PNG), imported into Unity	Fast model + image generation tool or asset packs
Sound Designer	Audio direction, UX feedback list	Sound effects and music (WAV, OGG), imported into Unity	Fast model + audio tool or asset packs
QA Playtester	Game build, game rules, balance file	Tests and the buyer/swapper simulation (C#), bug and balance reports (Markdown)	Fast, cheaper model

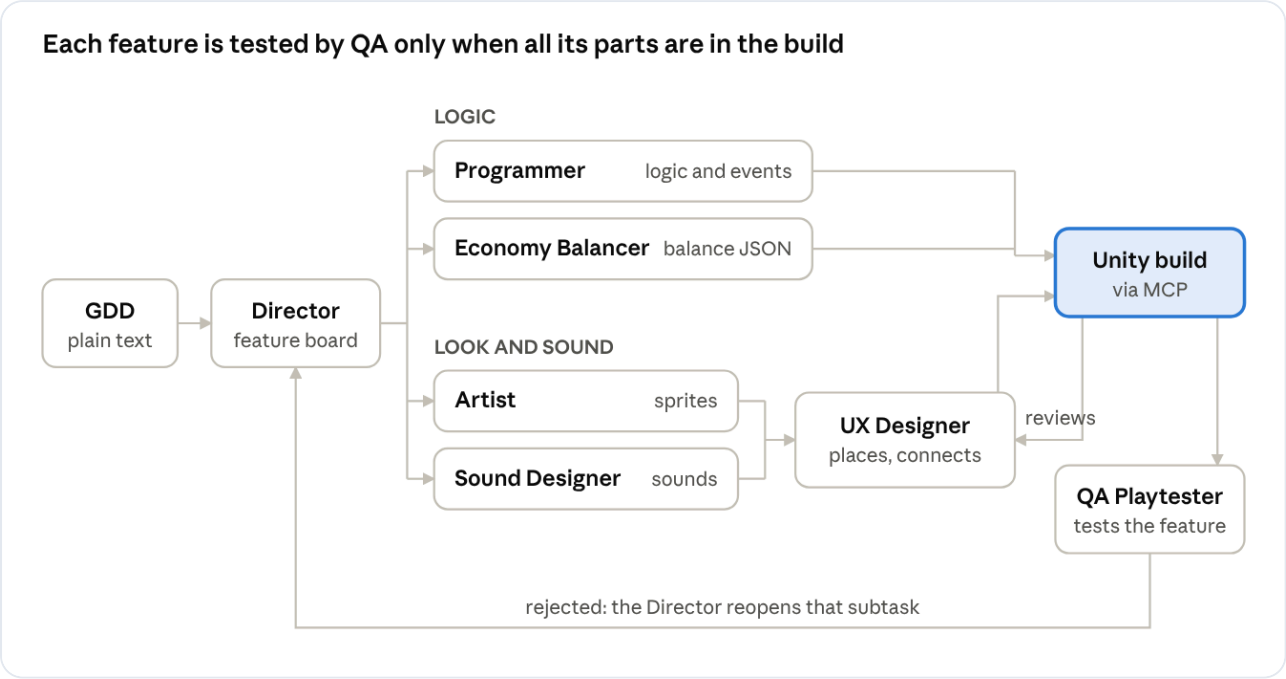
How they work together: the Director keeps a feature board, like Jira: one card per feature, with a subtask for each agent involved. When every subtask on a card is done, the QA Playtester tests the finished feature in the build. If it passes, the card is finished; if not, the Director reopens only the subtask that broke. The Economy Balancer and QA Playtester share one tool: a simulation that plays 7 days with two bots, a *buyer* that only buys new parts and a *swapper* that harvests parts between ships. If the buyer survives or the swapper goes bankrupt, the numbers go back to the Economy Balancer; after 3 failed tuning rounds in a row, the Director takes it to the designer instead of looping. The designer is the final decider on any rule the GDD leaves open: agents escalate through the Director and get an answer within the same work session. The UX Designer's feedback list is the shared plan for the look and sound of every event: the Artist and Sound Designer make what it asks for, and the UX Designer places it in the game.

Example: one feature card. Binning a part pays a little scrap money. On the Director's board this is one card with five subtasks. The Economy Balancer sets the value (10 credits) so it never beats a swap. The Programmer adds the bin logic and a "part binned" event. The Artist draws the bin and the coin, and the Sound Designer makes the clink. The UX Designer places them and connects them to the event: the part drops into the bin, a coin pops up, the clink plays and the credit counter ticks up. Then the QA Playtester tests the whole feature and checks that every binned part pays out exactly once and the credit counter adds up.

Technical Strategy

AI dev pipeline

The GDD is the only source the agents work from. The Director turns it into feature cards, and the work reaches the game along two paths: logic and data go straight into the Unity build, while art and sound go through the UX Designer, who places them in the game.



AI dev pipeline · from GDD text to the Unity build

The Artist and Sound Designer work from the UX Designer's feedback list, and the UX Designer reviews the build to ask for changes. A feature is finished only when the QA Playtester has tested it in the build.

Agent roles and access

All agents are defined in the two agent tables in AI Architecture. Each agent reads only what its task needs, which keeps token use low and stops agents from changing each other's work.

Prompt constraints

Every agent's prompt includes the same five rules, so outputs stay consistent from task to task:

1. Work only on your subtask from the feature board, and only in your own files and components.
2. Take every number from the GDD or the balance file; never invent values. The GDD's numbers are starting values: the Economy Balancer may change them to meet the balance targets, and says so in its task summary.
3. Use the agreed file names and output format, and check them yourself before submitting: C# for code, JSON for balance data, PNG for images, WAV or OGG for sound, Markdown for plans and reports.
4. If the GDD is unclear or two rules conflict, stop and report it to the Director instead of guessing.
5. End every task with a short summary of what changed, for the QA Playtester.

Engine integration

- Unity 2D project, PC build. Agents work in the editor through MCP.
- Code: C# scripts, one folder per system (bays, parts, inventory, economy, screens).
- Balance data: one JSON file with prices, payments, scrap value, required levels, damage patterns, junkyard contents and the penalty price, loaded when the game starts. Changing the balance never needs a code change.
- Art and sound: imported by the Artist and Sound Designer into fixed folders, under the names from the feedback list.
- Logic and visuals stay separate: the Programmer owns the game's events and the scene files, and the UX Designer's feedback only listens to those events from its own folder and visuals prefabs. Neither edits the other's files.
- A feature is finished only after the QA Playtester has tested it in the Unity build.
- Game rules live in plain C# classes, separate from scenes, so the simulation can run them quickly without the Unity editor.
- Version control: a GitHub repo. Agents working on the same feature share one feature branch. Each subtask lists the files it will change, and the Director never runs two subtasks on the same file at once.

Token budget

The MVP (first 2 weeks) is estimated at about **28 million tokens**, including a 25% rework reserve, and weeks 3-7 at about 10 million per week. A session is one task from start to accepted change; each of its steps resends the conversation so far, so it uses many times more tokens than the files it reads. After the first week, real use per session is measured and the budget is updated.

Agent	Sessions	Tokens per session	Total
Director	10	120,000	1.2M
Programmer	30	400,000	12.0M
Economy Balancer	10	160,000	1.6M
UX Designer	15	160,000	2.4M
Artist	10	80,000	0.8M
Sound Designer	5	80,000	0.4M
QA Playtester (incl. simulation)	20	200,000	4.0M
Rework reserve (25%)			5.6M
Total (MVP, 2 weeks)	**100**		**28.0M**

Cost: the build runs on the course's Claude Team subscription, so tokens cost no money directly; the real limit is the plan's session and weekly usage limits. At API prices (Sonnet 5.5 for the Director and Programmer, Haiku 5.5 for the rest, with prompt caching), the same work would cost roughly $25 for the MVP and $75 for the whole build. Unity Personal is free; art and audio tools or asset packs about $50-100.

Why it's plausible: the MVP is a small 2D game with eight screens, and a simple version was built in 2-3 days at a game jam. **Spending rule:** if a week goes over its budget or a usage limit is hit, the Director warns the designer, who decides what to postpone.

API constraints

1. **Context window.** An agent can't hold the whole project in memory at once. *Handling:* each agent gets only its GDD section and the files for its task, and all balance numbers live in one data file instead of being spread through the code.
2. **Rate and usage limits.** The API limits how many tokens can be used per minute and per day. *Handling:* agents work in parallel whenever their subtasks don't depend on each other, for example the Artist drawing while the Programmer writes the logic. If a limit is hit, the Director pauses the remaining subtasks and resumes them when the limit resets, starting with the ones other agents are waiting for.
3. **Agents can't judge fun.** The simulation proves the numbers work, but not whether swapping *feels* clever. *Handling:* the designer playtests at the end of every week, and those playtests decide the next balance changes.
4. **Engine choice.** Agents work best when everything is text they can read and edit. *Handling:* Unity with MCP. The designer has years of experience with Unity.

MVP scope

The MVP is done when a new player can play 7 days, discover part swapping, and want to play again. It is planned for the first 2 weeks of the 7-week build; the rest of the time goes to the features in After the MVP and to release. Everything is built by one developer (the designer) with the 7 agents.

In the MVP:

- 3 docking bays with ship arrival
- 2 ship types (fighter, freighter) with 4-5 part slots, using the same part types
- Condition bars per part, colored against the required level
- Move, bin, buy and release, with weak or empty slots filled on release
- Junkyard that refills each game day
- Shared inventory with 10 slots
- Credits, payment, daily upkeep, bankruptcy
- Two day types, announced at the start of each day: quiet day (ship count) and busy day (countdown)
- End-of-day summary
- Buyer/swapper simulation (a QA tool, not seen by the player)
- Optional guided tutorial (day 0) with the swapping hint, then a reset of credits and inventory; a fallback hint for players who skip it
- Day start screen with the day type, its reason and a junkyard button
- Run end screen with the run summary

Not in the MVP: station upgrades, improving parts and the other features in After the MVP; mobile repair ship, travel, story, competitors, more ship types, crafting and multiplayer go to the Expansion plan.

After the MVP

Weeks 3-7 add a few features in this order, then polish and release. Each one supports a design principle; anything that doesn't fit the time is moved to the Expansion plan.

Order	Feature	Supports	Weeks
1	Station upgrades bought with credits, for example a bigger inventory	Resourcefulness beats spending	3
2	Improving parts: raising a part's condition for credits	Good enough is good enough	3-4
3	Ships with more part types	Three ships, one puzzle	4
4	Endless mode, unlocked after the first successful run	Three ships, one puzzle	5
5	Time mode: finish a fixed number of ships as fast as possible. No time limit, but a slow finish gives a bad result	Three ships, one puzzle	5
6	Day news on the day start screen: a battle or a cosmic event hints which parts will be in demand	Resourcefulness beats spending	5-6
-	Polish, playtests and release	-	6-7

Not scheduled yet: a mobile repair ship that travels to stranded ships, instead of the static station.

Appendix: Revision notes (v1.2 and v1.3)

Version 1.1 went through the agent stress test (six AI reviewers, cross-examination and a synthesis) and the designer's own review. The verdict was "not ready to drive production": core rules were undecided, the game could soft-lock, and the economy could not reward swapping. Version 1.2 then went through a second stress test; the rows marked "second run" are the changes in v1.3. These are the changes:

Flagged by	Problem	Change in v1.2
Stress test (5 of 6 reviewers)	Core rules were undecided or contradicted each other, so agents told to "stop and report" would stall on day one. No human decider.	Decided: parts fit both ship types, required levels 60% and 70%, scrap value 10, day limits, drag-to-move, the swapping hint. The designer is named as the final decider.
Stress test and designer's own review	Soft-lock: a ship nobody can afford to fix freezes the day. (Spotted while writing v1.1 and left in on purpose, to see if the review would find it. It did.)	A ship can always be released: empty or too-weak slots are filled with new parts at the penalty price (20% above the shop price), after a warning.

Flagged by	Problem	Change in v1.2
Stress test	Parts are conserved: every ship takes in broken parts and needs good ones, so swapping alone can't beat buying. A buy-only player never went bankrupt.	Added the junkyard from the jam version: a cheap source of parts, most of them junk, always priced above scrap value. Ships now arrive with 1-3 weak parts.
Stress test	Discovering the swapping trick wasn't designed, and color bars misled in the 60-69% range.	Defined when the hint appears and how it sounds. Bars are now colored against each ship's required level. Day 0 became an optional guided tutorial, with a fallback hint for players who skip it.
Stress test	The buyer/swapper simulation, the only economy check, had no owner, no budget and no stop rule.	The QA Playtester owns it, it has a token budget, game rules run without the Unity editor, and after 3 failed tuning rounds the designer decides.
Own judgment	The mechanic's tone was undecided.	Proud and cheeky: the mechanic enjoys squeezing the most out of every part.
Assignment #2 criteria	Agent outputs had no formats; the one-person scope and the post-MVP weeks weren't stated.	File formats added to every agent; team stated as one developer plus 7 agents; new "After the MVP" section.
Designer's own review	With 500 starting credits and 50 upkeep, a player could survive 10 days without earning anything, longer than the 7-day run.	Upkeep raised to 120 per day.
Designer's own review	A player could hold back the last ship to avoid the end-of-day bankruptcy check.	Credits can go below 0 during the day and only the end-of-day balance counts, so holding a ship back gains nothing. Upkeep moved to the start of the day, so the counter always shows the real balance.
Stress test (feasibility lead, narrative critic, business analyst)	Parallel agents could overwrite each other; no rework in the budget; the post-MVP budget shrank while its work grew; the run had no ending; only token costs were counted.	Feature branches with one agent per file at a time; realistic per-session token estimates with a 25% rework reserve, measured after week 1; a run end screen; other costs added.
Stress test, second run (3 BLOCKING)	Swapping might not stay profitable on hard days; busy days could be beaten by stalling; the swapping trick had no reliable teaching path.	The GDD's numbers are starting values the Economy Balancer may change, with the target that a good swapper stays slightly positive also on the hardest days (profitable ships also make stalling pointless). The tutorial is guided and can't be lost, with a fallback hint for players who skip it.
Stress test, second run	Unclear money during the day, celebrating losing releases, undefined part moves, misleading colors, no ending for a loss, day types that didn't fit a static station, nobody writing the mechanic's lines.	Live credit counter with upkeep paid at the start of the day; feedback matches the quality of the move; swap on occupied slots and a buy button per slot; green for passing parts and a separate surplus color; a run end screen also after bankruptcy; quiet and busy days; the UX Designer writes the lines.
Stress test, third run	The tutorial hint called a 65% engine that fails the fighter's level "too good", and releasing a weak ship could still pay.	Hint rewritten; filling one weak part must cost more than the lowest ship payment.
Assignment #1 feedback	Close the open questions so the MVP stops having forks, and make the swap loop the one thing to polish.	All open questions are decided and the section is removed, including the last junkyard alternative. The swap loop stays the core of every system: junkyard, tutorial and economy targets.

Kept as is: the AI pipeline stays in this document, because the agents read it as their spec. Out of scope for this GDD: storefront, pricing, marketing and post-release plans.